Chapter - 9

Pointers, Virtual Functions and Polymorphism

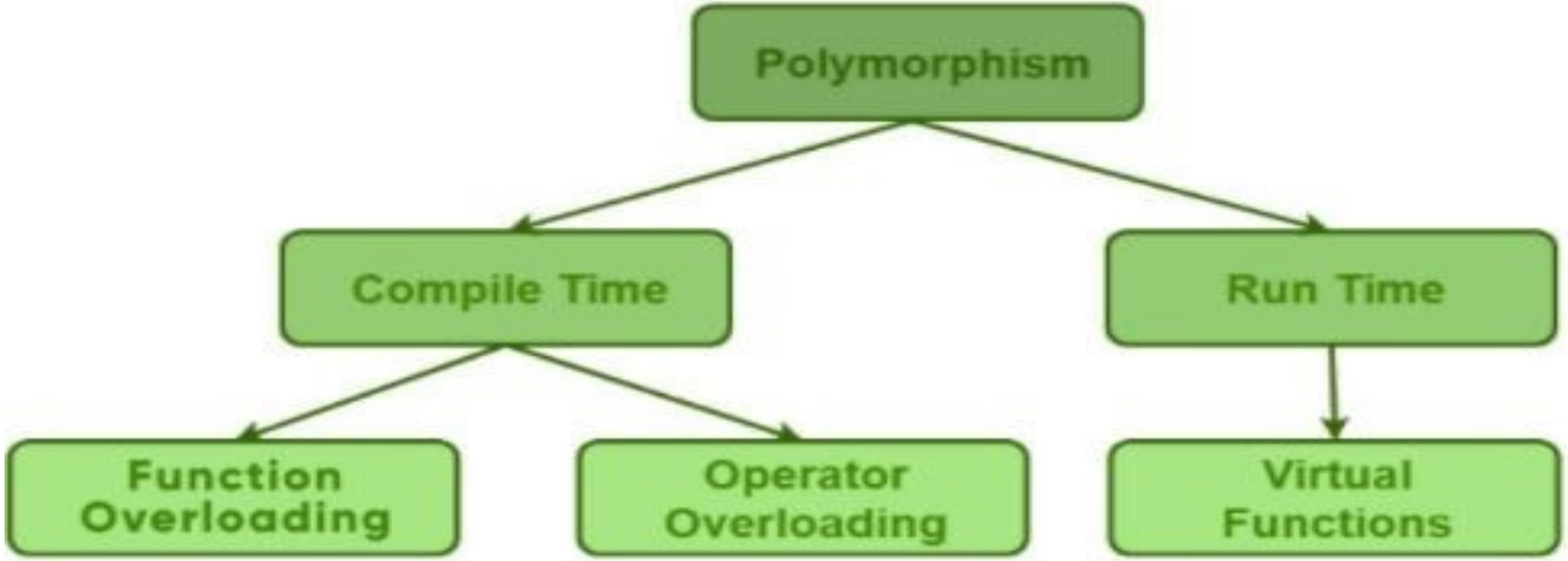
Book Referred: Object-Oriented Programming with C++ by E Balaguruswamy (8th Edition) – Chapter 9

Prepared By: Dr. Anuja Nair, Assistant Professor,
CSE Department, Nirma University

Concepts Included

- Polymorphism
- Pointers
- Pointers to objects
- this pointer
- Pointers to derived classes
- Virtual functions
- Pure virtual function

Polymorphism



- Compile time polymorphism → early binding/static binding/static linking
- Run time polymorphism → late binding/dynamic binding → requires pointers to objects

Pointers

- Declaring and initializing pointers
- Manipulation of pointers
- Pointer expressions and pointer arithmetic

Using pointers with Arrays and Strings

- We can declare the pointers to arrays as follows:
  - int *ptr;
  - ptr = number[0]; OR
  - ptr = number;
  - Here, ptr points to the first element of the integer array, number[0].
  - ptr++; or ++ptr; or ptr--; or --ptr;
- In C++, there is no automatic array bounds checking.
- Suppose we declare an array of size 25. The compiler issues no warnings if we attempt to access 26th location. It is the programmer's task to check the array limits.

Refer 9a.cpp

Array of Pointers

- Similar to other variables, we can create an array of pointers in C++.
- The array of pointers represents a collection of addresses. By declaring array of pointers, we can save substantial amount of memory space.
- An array of pointers point to an array of data items.
- Each element of the pointer array points to an item of the data array. Data items can be accessed either directly or by dereferencing the elements of pointer array.
- We can declare an array of pointers as follows:
  - int *inarray[10];

Refer 9b.cpp

Pointers and Strings

- In C++, a string (C-style string) is stored as an array of characters ending with a null character '\0'.
  - char str[] = "hello";
- Memory Representation:
  - h e l l o \0
- A pointer can be used to point to a string because a string's name itself acts like an address of the first character.

1. Pointer to a String Literal

- char *p = "hello";
  - "hello" is stored in read-only memory
  - p stores the address of 'h'
- So, p points to the whole string.
  - p → h e l l o \0
- But we should not modify this string, because it's read-only:
  - p[0] = 'H'; // Error (undefined behavior)

Pointers and Strings

Refer 9c.cpp

2. Pointer to a Character Array
   - char str[] = "hello";
   - char *p = str;

- Here,
  - str is an actual **mutable array** in memory.
  - p points to the **same array**, so modification is allowed.
- p[0] = 'H'; // Valid
- str → H e l l o \0

p ——┘

Pointers to Functions

- Just like we can store the address of a variable in a pointer, we can also store the address of a function in a pointer.
- This lets us:
  - Call a function using a pointer
  - Pass functions as arguments to other functions
- Syntax: return_type (*pointer_name)(parameter_list);
- Example: int (*p)(int, int);
- This means p is a pointer pointing to a function that returns int and takes two integers as parameters.

Refer 9d.cpp and 9e.cpp

Pointer to object

- A pointer to an object is a pointer that holds the memory address of an object. Using pointers to objects allows for dynamic memory allocation, polymorphism, and efficient manipulation of objects in programs.
- **Syntax:**
  - ClassName* ptr;
  - ptr is a pointer that can store the address of an object of type ClassName.

Refer 9f.cpp

Example

```
#include <iostream>

class MyClass {
public:
    void display() {
        std::cout << "Hello, I am an object!" << std::endl;
    }
};

int main() {
    MyClass obj;            // Create an object
    MyClass* ptr = &obj;    // Pointer to the object

    // Accessing members using the pointer
    ptr->display();         // Use '->' to call a member function

    return 0;
}
```

```
Hello, I am an object!
```

Pointers can be used to dynamically allocate memory for objects using the **new** keyword.

```
#include <iostream>

class MyClass {
public:
    void display() {
        std::cout << "Dynamic object!" << std::endl;
    }
};

int main() {
    // Dynamically allocate memory for an object
    MyClass* ptr = new MyClass();

    // Access member function using pointer
    ptr->display();

    // Free dynamically allocated memory
    delete ptr;

    return 0;
}
```

```
Dynamic object!
```

this pointer

- The this pointer is an implicit pointer available in all non-static member functions of a class. It points to the object for which the member function is called.
- The this pointer is particularly useful when you need to refer to the current object, especially when distinguishing between class members and local variables or parameters with the same name.
- Scope: It is available in non-static member functions of a class. Static member functions do not have access to the this pointer because they are not tied to any specific object.
- Type: The type of the this pointer is a pointer to the class type (ClassName*), and it points to the memory address of the object invoking the member function.

this pointer

- In C++, every object has its own address in memory. Inside a member function, the keyword this stores the address of the object that is calling the function.
  - this → pointer to the current object
- It is automatically provided by the compiler - you do not need to declare it.
- To distinguish between:
  - Member variables (data members)
  - Local variables or function parameters with same names

```cpp
class Test {
    int a;
public:
    void set(int a) {
        a = a;   // assigns parameter a to itself; data member
            unchanged
    }
};
```

```cpp
class Test {
    int a;
public:
    void set(int a) {
        this->a = a;   // assigns parameter a to data member a
    }
};
```

this pointer

- One another important application of the pointer this is to return the object it points to. For example, the statement
  - return *this;
- inside a member function will return the object that invoked the function. This statement assumes importance when we want to compare two or more objects inside a member function and return the invoking object as a result.

Refer 9g.cpp and 9h.cpp

Accessing Members using this pointer

The this pointer allows access to the invoking object's members directly.

```cpp
#include <iostream>
#include <string>
class Student {
    std::string name;
    int age;
public:
    // Constructor to initialize student details
    Student(const std::string& name, int age) {
        this->name = name;  // Using the 'this' pointer to differentiate class member and parameter
        this->age = age;
    }
    // Method to update student details
    Student& updateDetails(const std::string& newName, int newAge) {
        this->name = newName;  // Using the 'this' pointer
        this->age = newAge;
        return *this;          // Returning the current object for method chaining
    }
    // Method to display student details
    void display() const {
        std::cout << "Name: " << this->name << ", Age: " << this->age << std::endl;
    }
};
int main() {
    Student student("Aayushi", 18);
    std::cout << "Initial Details:" << std::endl;
    student.display();
    std::cout << "\nUpdating Details:" << std::endl;
    student.updateDetails("Nishat", 15).display();
    return 0;
}
```

```
Initial Details:
Name: Aayushi, Age: 18

Updating Details:
Name: Nishat, Age: 15
```

Pointers to Derived Classes

- If class D is derived from class B, then:
  - B *ptr; // pointer of base class
  - D d; // object of derived class
  - ptr = &d; // base class pointer can point to derived object
- This is allowed because a derived object contains all members of the base class.
- <u>But there is a limitation.</u>
- When we use a base class pointer (B*) to refer to a derived object (D):
  - We can access only the base class members
  - We cannot access the members added in the derived class
- Why? Because the type of the pointer decides what members are accessible, not the object it points to.

Pointers to Derived Classes

- Even though d has showDerived(), we cannot call it using B* pointer.
- So to access derived class members, we must use a derived class pointer:
- D *dp = &d;
- dp->showDerived(); // works

```
class B {
public:
    void showBase() { cout << "Base class\n"; }
};

class D : public B {
public:
    void showDerived() { cout << "Derived class\n"; }
};

int main() {
    D d;
    B *ptr = &d;  // base pointer pointing to derived object

    ptr->showBase();     // allowed
    ptr->showDerived();  // not allowed (compile-time error)
}
```

Virtual Functions

- There is a necessity to use the **single pointer to refer to all the objects of the different classes.**

- So, we create the pointer to the base class that refers to all the derived objects.

- But, when **base class pointer contains the address of the derived class object, always executes the base class function.**

- This issue can only be resolved by using the **'virtual' function.**

Virtual Functions

- Virtual function is a member function in the base class that you redefine in a derived class. It is declared using the **virtual keyword**.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- It is used **to tell the compiler to perform dynamic linkage or late binding on the function**.
- When the function is made virtual, C++ determines which function is to **be invoked at the runtime** based on the **type of the object** pointed by the base class pointer and **not by the type of the pointer.**
- In late binding function call is resolved during **runtime**. Therefore compiler determines the type of object at runtime, and then binds the function call.

Virtual Functions

```
class BC
{
public:
    void display()
    {
        cout<<"\n Display base";
    }
    virtual void show()
    {
        cout<<"\n Show base";
    }
};

class DC : public BC
{
public:
    void display()
    {
        cout<<"\n Display derived";
    }
    void show()
    {
        cout<<"\n Show derived";
    }
};
```

```
int main()
{
    BC *bptr;
    BC B;
    DC D;

    cout << "\n bptr points to Base \n";
    bptr = &B;
    bptr ->display(); //calls Base Version
    bptr -> show(); //calls Base Version

    cout << "\n\n bptr points to Derived\n";
    bptr = &D;
    bptr ->display(); //calls Base Version
    bptr -> show(); //calls Derived Version
}
```

```
bptr points to Base

Display base
Show base

bptr points to Derived

Display base
Show derived
```

Virtual Functions

NOTE:

- Run time polymorphism is achieved only when a virtual function is accessed through a pointer to the base class.

Rules of Virtual Function

1. Virtual functions must be members of some class.
2. Virtual functions cannot be static members.
3. They are accessed through object pointers.
4. They can be a friend of another class.
5. A virtual function must be defined in the base class, even though it is not used.
6. The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions and the virtual function mechanism is ignored
7. We cannot have a virtual constructor, but we can virtual destructors.

Rules of Virtual Function

8. While a base pointer can point to any type of the derived object, the reverse is not true. That is to say, we can not use a pointer to a derived class to access an object of base type
9. When a base pointer points to a derived class, incrementing or decrementing it will not make it to point to the next object of the derived class. It is incremented or decremented only relative to it's base type. Therefore, we should not use this method to move the pointer to the next object.

Virtual Destructors

- A **virtual destructor** ensures that the correct **destructor of the derived class** is called when an object is deleted through a **base class pointer**.
- Without a virtual destructor, **only the base class destructor** runs, and this can lead to **resource leaks** or **incomplete destruction** of derived objects.

🔑 **Rule of Thumb**

If your class has **virtual functions**, **always make the destructor virtual**, especially if it will be **inherited**.

Without Virtual Destructors

```cpp
#include <iostream>
using namespace std;

class Base
{
public:
    Base() { cout << "Base constructor\n"; }
    ~Base() { cout << "Base destructor\n"; }
};

class Derived : public Base
{
public:
    Derived() { cout << "Derived constructor\n"; }
    ~Derived() { cout << "Derived destructor\n"; }
};

int main()
{
    Base* b = new Derived();  // Base pointer to Derived object
    delete b;                 // Will call only Base destructor
    return 0;
}
```

```
Base constructor
Derived constructor
Base destructor
```

With Virtual Destructors

```cpp
#include <iostream>
using namespace std;

class Base
{
public:
    Base() { cout << "Base constructor\n"; }
    virtual ~Base() {cout<<"Base destructor\n"; }// Virtual
        destructor
};

class Derived : public Base
{
public:
    Derived() { cout << "Derived constructor\n"; }
    ~Derived() { cout << "Derived destructor\n"; }
};

int main()
{
    Base* b = new Derived();  // Base pointer to Derived object
    delete b;       // Proper destruction with virtual destructor
    return 0;
}
```

```
Base constructor
Derived constructor
Derived destructor
Base destructor
```

Practical : Virtual function

Create a **class Media** that stores the **title** (a string) and **price** (float). Class Media has two argument **constructor** which initializes data members of class Media. Also declare a virtual function **display** () in Class Media. From the class Media derive two classes: **Class book**, which contains data member **page count** (int): and **Class tape**, which contains data member **playing time** in minutes (float). Both Class book and Class tape should have a **constructor** which initializes base class constructor as well as its own data members and **display** ( ) function which displays book details and tape details respectively. Write a main ( ) to test book and tape classes by creating instances of them, asking the user to fill data and displaying them.

Use the concept of **Virtual function and Constructor in**

Practical : Virtual function

```
class media
{
protected:
    char title[50];
    float price;
public:
    media(char *s, float a)
    {
        strcpy(title, s);
        price=a;
    }
    virtual void display() {} //empty virtual funtion
};

class book: public media
{
    int pages;
public:
    book(char *s, float a, int p):media(s,a)
    {
        pages=p;
    }
    void display();
};
```

Practical : Virtual function

```
class tape :public media
{
    float time;
public:
    tape(char * s, float a, float t):media(s, a)
    {
        time = t;
    }
    void display();
};

void book :: display()
{
    cout<<"\n Title: "<<title;
    cout<<"\n Pages: "<<pages;
    cout<<"\n Price: "<<price;
}

void tape :: display()
{
    cout<<"\n Title: "<<title;
    cout<<"\n play time: "<<time<<"mins";
    cout<<"\n price: "<<price;
}
```

Practical : Virtual function

```
int main()
{
    char * title=new char[30];
    float price, time;
    int pages;

    cout<<"\n ENTER BOOK DETAILS\n";
    cout<<"Title: "; cin>>title;
    cout<<"Price: "; cin>>price;
    cout<<"Pages: "; cin>>pages;
    book book1(title, price, pages);

    cout<<"\n ENTER TAPE DETAILS\n";
    cout<<"Title: "; cin>>title;
    cout<<"Price: "; cin>>price;
    cout<<"Play time (mins): "; cin>>time;
    tape tape1(title, price, time);

    media* list[2];
    list[0]=&book1;
    list[1]=&tape1;

    cout<<"\n MEDIA DETAILS";
    cout<<"\n.......BOOK......";
    list[0] -> display(); //display book details
    cout<<"\n.......TAPE......";
    list[1] -> display(); //display tape details
}
```

```
 ENTER BOOK DETAILS
Title: ANSI_C
Price: 88
Pages: 400

 ENTER TAPE DETAILS
Title: OOPC
Price: 90
Play time (mins): 55

 MEDIA DETAILS
.......BOOK......
 Title: ANSI_C
 Pages: 400
 Price: 88
.......TAPE......
 Title: OOPC
 play time: 55mins
 price: 90
```

Size of Virtual function

```
#include<iostream>
using namespace std;

class A
{

};

class B
{
    int i;
};

class C
{
    void foo();
};

class D
{
    virtual void foo();
};
```

```
class E
{
    int i ;
    virtual void foo();
};

class F
{
    int i;
    void foo();
};

class G
{
    void foo();
    int i;
    void foo1();
};

class H
{
    int i ;
    virtual void foo();
    virtual void foo1();
};
```

Size of Virtual function

```
int main()
{
    cout <<"sizeof(class A) : " << sizeof(A) << endl ;
    cout <<"sizeof(class B) adding the member int i : " << sizeof(B) << endl ;
    cout <<"sizeof(class C) adding the member void foo() : " << sizeof(C) << endl ;
    cout <<"sizeof(class D) after making foo virtual : " << sizeof(D) << endl ;
    cout <<"sizeof(class E) after adding foo virtual , int : " << sizeof(E) << endl ;
    cout <<"sizeof(class F) after adding foo  , int : " << sizeof(F) << endl ;
    cout <<"sizeof(class G) after adding foo  , int : " << sizeof(G) << endl ;
    cout <<"sizeof(class H) after adding int 2 virtual " << sizeof(H) << endl ;
    return 0;
}
```

```
sizeof(class A) : 1
sizeof(class B) adding the member int i : 4
sizeof(class C) adding the member void foo() : 1
sizeof(class D) after making foo virtual : 8
sizeof(class E) after adding foo virtual , int : 16
sizeof(class F) after adding foo  , int : 4
sizeof(class G) after adding foo  , int : 4
sizeof(class G) after adding foo  , int : 4
sizeof(class H) after adding int 2 virtual 16
```

Abstract class & Pure Virtual Functions

- It is normal practice to declare a function virtual inside the base class and redefine it in the derived classes.
- Such function inside base class is not used for performing any task, it only serves as a placeholder.
- For ex, display() function in class Media in previous example. Such functions are called "do-nothing" functio
- A "**do-nothing**" function may be defined as follows:

- Such functions

```
virtual void display()=0;
```

s

Abstract class & Pure Virtual Functions

- A **pure virtual function** is a function declared in a base class that has **no definition** relative to the base class.

- In such case compiler requires each derived class to either define the function or redeclare it as a pure virtual function.

- Class containing pure virtual functions cannot be used to declare any objects of its own and such classes are called **abstract base classes**

Abstract class & Pure Virtual Functions

```
class B
{
public:
    virtual void s() = 0; // Pure Virtual Function
};

class D:public B
{
public:
    void s()
    {
        cout << "Virtual Function in Derived class\n";
    }
};

int main()
{
    B *b;
    D dobj;
    b = &dobj;
    b->s();
}
```

```
Virtual Function in Derived class
```

Abstract class & Pure Virtual Functions

NOTE:

1. **A class is abstract if it has at least one pure virtual function.**

```
#include<iostream>
using namespace std;

class Test
{
   int x;
public:
    virtual void show() = 0;
    int getX() { return x; }
};

int main(void)
{
    Test t;   //ERROR
    return 0;
}
```

```
Compiler Error: cannot declare variable 't' to be of abstract
 type 'Test' because the following virtual functions are pure
within 'Test': note:     virtual void Test::show()
```

2. We can have pointers and references of abstract class type.

```
class Base
{
public:
    virtual void show() = 0;
};

class Derived: public Base
{
public:
    void show() { cout << "In Derived \n"; }
};

int main(void)
{
    Base *bp = new Derived();
    bp->show();
    return 0;
}
```

```
In Derived
```

3. If we do not override the pure virtual function in derived class, then derived class also becomes abstract class.

```
#include<iostream>
using namespace std;
class Base
{
public:
    virtual void show() = 0;
};

class Derived : public Base { };

int main(void)
{
  Derived d;   //ERROR
  return 0;
}
```

```
Compiler Error: cannot declare variable 'd' to be of abstract type
'Derived'  because the following virtual functions are pure within
'Derived': virtual void Base::show()
```

4. An abstract class can have constructors.

```
// An abstract class with constructor
class Base
{
protected:
    int x;
public:
    virtual void fun() = 0;
    Base(int i)
    {
        x = i;
        cout<<"Constructor of base called\n";
    }
};

class Derived: public Base
{
    int y;
public:
    Derived(int i, int j):Base(i)
    {
        y = j;
    }
    void fun()
    {
        cout << "x = " << x << ", y = " << y<<'\n';
    }
};
```

```
int main(void)
{
    Derived d(4, 5);
    d.fun();

    Base *ptr=new Derived(6,7);
    ptr->fun();
    return 0;
}
```

```
Constructor of base called
x = 4, y = 5
Constructor of base called
x = 6, y = 7
```

Practical :Abstract class & Pure Virtual Functions

Create an Abstract class **vehicle** having **average** as data and pure virtual function **getdata()** and **putdata().** Derive class **car** and **truck** from class vehicle having data members: **fuel type** (petrol, diesel, CNG) and **no of wheels** respectively. Write a main ( ) that enters the data of two cars and a truck and display the details of them.

Use the concept of **Abstract Base class** and **Pure Virtual functions.**

Practical :Abstract class & Pure Virtual Functions

```
class vehicle
{
protected:
    float avg;
public:
    virtual void getdata()=0;
    virtual void putdata()=0;
};
class car: public vehicle
{
    char ftcar[10];
    int nw_car;
public:
    void getdata()
    {
        cout<<"Enter Average of Car: ";        cin>>avg;
        cout<<"Enter Fuel-Type for car: ";     cin>>ftcar;
        cout<<"Enter No of wheels for car : "; cin>>nw_car;
    }
    void putdata()
    {
        cout<<" Average of Car is: "<<avg<<endl;
        cout<<"Fuel-Type for car is : "<<ftcar<<endl;
        cout<<"No of wheels for car is : "<<nw_car<<endl;
    }
};
```

Practical :Abstract class & Pure Virtual Functions

```
class truck: public vehicle
{
    char fttruck[10];
    int nw_truck;
public:
    void getdata()
    {
        cout<<"Enter Average of Truck: ";
        cin>>avg;
        cout<<"Enter Fuel-Type for Truck: ";
        cin>>fttruck;
        cout<<"Enter No of wheels for Truck : ";
        cin>>nw_truck;
    }
    void putdata()
    {
        cout<<" Average of Truck is: "<<avg<<endl;
        cout<<"Fuel-Type for Truck is : "<<fttruck<<endl;
        cout<<"No of wheels for Truck is : "<<nw_truck<<endl;
    }
};
```

Practical :Abstract class & Pure Virtual Functions

```
int main()
{
    vehicle *list[2];
    car c1;
    truck t1;
    list[0]=&c1;
    list[1]=&t1;
    cout<<"Enter Car Details: "<<endl;
    list[0]->getdata();
    cout<<"Enter Truck Details: "<<endl;
    list[1]->getdata();
    cout<<".........Car Details........."<<endl;
    list[0]->putdata();
    cout<<".........Truck Details........."<<endl;
    list[1]->putdata();
}
```

```
Enter Car Details:
Enter Average of Car: 23.2
Enter Fuel-Type for car: Petrol
Enter No of wheels for car : 4
Enter Truck Details:
Enter Average of Truck: 45.78
Enter Fuel-Type for Truck: CNG
Enter No of wheels for Truck : 8
.........Car Details.........
 Average of Car is: 23.2
Fuel-Type for car is : Petrol
No of wheels for car is : 4
.........Truck Details.........
 Average of Truck is: 45.78
Fuel-Type for Truck is : CNG
No of wheels for Truck is : 8
```

Thank You